

14. Bölüm

Jenerikler, Koleksiyonlar ve İndisleyiciler

Şu ana kadar en basit veri yapıları (**data structure**) olan değişkenlerle karşılaştık. Programlamada en çok kullanılan bir başka veri yapısı da dizilerdir (**array**). Diziler de koleksiyonlardan (**collection**) biridir ve daha önce anlatılmıştır. Koleksiyonlar çok sayıda nesneyi barındıran dinamik veri yapıları olup **new** anahtar kelimesiyle çalışma anında (**run-time**) bellekte oluşturulan yapılardır.

14.1 ArrayList

Dizilerin çalışma anında boyutu belirlenir ve daha sonra boyutu değiştirilmez. Aksine boyutu dinamik olarak artırılan bir dizi kullanmak istediğimizde **ArrayList** kullanırız. **ArrayList**, .NET için standart olarak tanımlı **ICollection** arayüzünü (**interface**) uygulayan bir koleksiyondur.

using System.Collections;

```
ArrayList liste = new ArrayList();
liste.Add("Renault");
liste.Add("Fiat");
liste.Add("Toyota");
Console.WriteLine("Araçlar");
Console.WriteLine("    Adet:    {0}", liste.Count); //3
Console.WriteLine("    Kapasite: {0}", liste.Capacity); //4
Console.WriteLine("    Değerler:");
foreach (Object obj in liste)
    Console.WriteLine("    {0}", obj); // Renault Fiat Toyota
```

ArrayList Kullanmaktan Kaçınma

ArrayList içindeki verileri **object** olarak sakladığı için değer tipleriyle (**int**, **double** vb.) çalışırken kutulama (**boxing/unboxing**) maliyetine yol açar. Bu nedenle modern uygulamalarda ileride anlatılan **List<T>** kullanılır.

Nesneleri bir listede saklamak için eskiden **ArrayList** kullanırdık. **ArrayList** Her şeyi **object** olarak tutar. İçine **int** eklediğinizde her defasında kutulama (**boxing**) yapar. Okurken de kutudan çıkarma (**unboxing**) yapar. Bu yöntem **çok yavaştır!** Artık bu işlemi aşağıda anlatılacak **List<int>** ile yapıyoruz. **List<int>** tip güvenlidir (**type-safe**). Verileri doğrudan kendi tipinde tutar. Kutulama (**boxing**) yapmaz ve hızlıdır.

14.2 Jenerik Programlama

Jenerik programlamada (**generic programming**) jenerik yapılar, kodun tipten bağımsız olarak yazılmasını, ancak kullanım anında tip güvenliğinin (**type safety**) korunmasını sağlar. Bu sayede aynı mantığı **int**, **string** veya kullanıcı tanımlı sınıflar için tekrar tekrar yazmak zorunda kalmazsınız.

Jenerik yapılar gelmeden önce (**List<T>** yerine **ArrayList** varken), her şey **object** olarak tutulurdu. Bu da her işlemde kutulama (**boxing**) ve kutuyu açma (**unboxing**) maliyetine ve tip hatalarına yol açardı. Jenerik yapılar bu sorunu kökten çözer.

C# dili farklı veri tipleriyle çalışan yöntemler, sınıflar, algoritmaları tekrar tekrar yazmamak için bir programlama türü olan **jenerik programlamayı** (**generic programming**) destekler. **Dizi** ve **ArrayList** dışındaki tüm koleksiyonlar jenerik programa ile çalışır.

Jenerikler (**generic**), veri tiplerine bağlı olmadan algoritma içeren bir yöntem veya sınıf oluşturmamızı sağlar! Jenerik, aynı mantığı yeniden yazmadan bir yöntemin farklı veri tipleri üzerinde çalışmasını sağlayan bir sınıf, yöntem ve algoritma planı tanımlar. Jenerikler, kodunuzda sınıfı veya yöntemi kullanana kadar bir veya daha fazla tip parametresinin belirtilmesini erteleyen sınıflar ve yöntemler tasarlamayı mümkün kılar.

Genel bir veri tipini temsil eden **T** veya **V** parametresi kullanarak, çalışma zamanı dönüştürmelerinin veya kutulama (**boxing**) işlemlerinin maliyetine veya riskine katlanmadan diğer istemci kodunun kullanabileceği tek bir sınıf yazabilirsiniz;

```
GenericList<int> list1 = new(); //int veri tipinde elemanlardan oluşan liste
list1.Add(1);
```

```
GenericList<string> list2 = new(); //string veri tipinde elemanlardan oluşan liste
list2.Add("");
```

```
GenericList<Aclass> list3 = new(); //Aclass veri tipinde elemanlardan oluşan liste
list3.Add(new Aclass());
```

```
public class GenericList<T>
{
    public void Add(T item) { }
}
```

```
public class Aclass { } // Örnek Bir Sınıf
```

Yukarıdaki örnek programda tanımladığımız jenerik olan `GenericList<T>`, hem `int` veri tipiyle, hem `string` veri tipiyle hem de kullanıcı tanımlı `Aclass` veri tipiyle çalışmaktadır.

§14.2.1 Jenerik Sınıf Tanımlama

C++'taki `template <typename T>` ifadesinin yerini C# dilinde `<T>` alır.

```
public class Depo<T>
{
    private T _icerik;

    public void Ekle(T veri) => _icerik = veri;
    public T Getir() => _icerik;
}
```

// Kullanımı:

```
Depo<int> sayiDeposu = new Depo<int>();
sayiDeposu.Ekle(100);
Depo<string> metinDeposu = new Depo<string>();
metinDeposu.Ekle("C# Harika!");
```

§14.2.2 Jenerik Kısıtlamalar

C++ şablonlarında bir tipin belirli özellikleri karşılamadığını ancak derleme hatası aldığınızda anlarsınız. C# dilinde ise `where` anahtar kelimesiyle en baştan kurallar koyabilirsiniz. Bu **kısıtlamalar** (**constraints**) için `where` şartı yazılır. Bu, jenerik programlamanın en güçlü yanlarından biridir.

```
public class MatematikIslem<T> where T : struct // T mutlaka bir değer tipi (int, double vb.)
→ olmalı
{
    // ...
}
public class VeriKaydedici<T> where T : class, new() // T bir sınıf olmalı ve parametresiz yapıcı
→ metodu olmalı
{
    public T YeniNesneOlustur() => new T();
}
```

Sık kullanılan kısıtlamalar:

- ◊ `where T : struct` → Değer tipi zorunluluğu.
- ◊ `where T : class` → Referans tipi zorunluluğu.
- ◊ `where T : new()` → T sınıfından varsayılan yapıcıyla bir örnek olmalı.
- ◊ `where T : NotNull` → `null` olamaz.
- ◊ `where T : ASınıfı` → Belirli bir sınıftan türemiş olmalı.
- ◊ `where T : IArayüzü` → Belirli bir arayüzü uygulamış olmalı.

§14.2.3 Jenerik Yöntemler

Sadece sınıflar değil, yöntemler da jenerik olabilir. Tip çıkarımı (**type inference**) sayesinde çoğu zaman tipi belirtmenize bile gerek kalmaz.

```
public void Takas<T>(ref T a, ref T b)
{
    T gecici = a;
    a = b;
    b = gecici;
}

// Kullanımı:
int x = 5, y = 10;
Takas(ref x, ref y); // Derleyici T'nin int olduğunu anlar.
```

§14.2.4 C++ Şablonları ve C# Jenerikleri

Karşılaştırmalı tablo aşağıda verilmiştir;

Özellik	C++ Templates	C# Jenerikleri
Derleme	Her tip için ayrı kod üretilir (code bloat).	Tek bir kod üretilir, çalışma zamanında özelleşir.
Tip Kontrolü	Gevşek (Kullanılana kadar hata vermez).	Sıkı (Kısıtlamalarla önceden belirlenir).
Statik Alanlar	Her tip örneği için ayrı statik alan oluşur.	Her tip örneği için ayrı statik alan oluşur.
Uzmanlaşma	<code>template<></code> ile belirli tiplere özel kod yazılabilir.	Doğrudan desteklenmez (Arayüzlerle çözülür).

Tablo 14.1: C++ Şablonu ve C# Jenerik Karşılaştırması

§14.2.5 Numaralandırıcılar ve Koleksiyonlar

C# dilinde bir nesnenin `foreach` döngüsü ile taranabilmesi için o nesnenin "sayılabilir" olması gerekir. Bu yetenek, .NET ekosisteminde iki temel arayüz ile sağlanır. Bunlar `IEnumerable` ve `IEnumerator` arayüzleridir.

1. `IEnumerable<T>` (Kumanda Edilebilir): Bir nesnenin "üzerinde gezilebilir" olduğunu beyan eder. Tek bir metodu vardır: `GetEnumerator()`. Bu, size bir "numaralandırıcı" (kumanda) verir.
2. `IEnumerator<T>` (Kumanda): Koleksiyon üzerinde adım adım ilerlemeyi sağlayan asıl mekanizmadır. O an hangi elemandayız? Bir sonraki eleman var mı? Bu soruların cevabını verir.

`IEnumerator<T>` Arayüzünün Anatomisi

Bir `IEnumerator<T>` arayüzü şu üç temel üyeye sahiptir:

1. `Current` Özelliği: İmlecin o an üzerinde bulunduğu elemanı döndürür.
2. `MoveNext()` Yöntemi: İmleci bir sonraki elemana kaydırır. Eğer bir sonraki eleman varsa `true`, listenin sonuna gelindiye `false` döner.
3. `Reset()` Yöntemi: İmleci başlangıç noktasına (ilk elemanın öncesine) geri getirir.

Aşağıda bu arayüzün kodu verilmiştir;

```
public interface IEnumerator<out T> : IDisposable, IEnumerator
{
    T Current { get; }
    bool MoveNext();
    void Reset();
}
```

`foreach` Döngüsünün Arka Planı

Programcı `foreach` yazdığında, C# derleyicisi bunu arka planda bir `while` döngüsüne ve `IEnumerator` kullanımına dönüştürür.

Programcının Yazdığı:

```
foreach (var item in liste) { Console.WriteLine(item); }
```

Sadeleştirilmiş Şekilde Derleyicinin Çalıştığı:

```
IEnumerator<int> enumerator = liste.GetEnumerator();
try {
    while (enumerator.MoveNext()) {
        int item = enumerator.Current;
        Console.WriteLine(item);
    }
}
```

```

}
finally {
    enumerator.Dispose();
}

```

Neden Doğrudan IEnumerator Kullanmıyoruz?

IEnumerable, listenin durumunu (**state**) korumaz. Her **GetEnumerator()** çağrıldığında yeni bir imleç oluşturulur. Bu sayede aynı listeyi aynı anda iki farklı **foreach** döngüsü ile (birbirinden bağımsız yerlerde) tarayabilirsiniz. Eğer sadece **IEnumerator** olsaydı, bir döngü ilerlediğinde diğeri de kaldığı yerden devam etmek zorunda kalırdı.

Bir **foreach** döngüsü ile bir listeyi dönerken, döngü içinde **liste.Add()** veya **liste.Remove()** yaparsanız ne olur? **InvalidOperationException** hatası alırsınız. **IEnumerator**, oluşturulduğu andaki koleksiyonun bir "versiyon" numarasını tutar. Koleksiyonda herhangi bir değişiklik yapıldığında bu versiyon numarası güncellenir. **MoveNext()** her çağrıldığında versiyon kontrolü yapılır; eğer numaralar eşleşmiyorsa koleksiyonun değiştiği anlaşılır ve hata fırlatılır.

§14.2.6 Yineleyiciler

Bu başlık sonrasında işleyeceğimiz koleksiyonlarda çok kullanılan yineleyiciler (**iterator**), **IEnumerable** ve **IEnumerator<T>** arayüzlerini kullanır.

Aşağıda bu arayüzleri uygulayan sınıflara örnek verilmiştir;

```

using System.Collections;
foreach (var araç in new AraçKoleksiyonu())
{
    Console.WriteLine($"{araç} "); //sedan heçbek siteyşın limuzin
}

public class AraçTipiNumaralandırma : IEnumerator<string> //IEnumerator Interface Realization
{
    string[] araçTipleri = new string[4] { "sedan", "heçbek", "siteyşın", "limuzin" };
    int currentIndex = -1;
    public string Current
    {
        get
        {
            if (currentIndex < 0 || currentIndex >= araçTipleri.Length)
                throw new InvalidOperationException();
            return araçTipleri[currentIndex];
        }
    }
    object IEnumerator.Current // Explicit implementation of IEnumerator.Current
    {
        get
        {
            return Current;
        }
    }
    public void Dispose()
    {
    }
    public bool MoveNext()
    {
        currentIndex++;
        if (currentIndex < araçTipleri.Length)
        {
            return true;
        }
        return false;
    }
    public void Reset()
    {
        currentIndex = 0;
    }
}

```

```

    }
}
public class AraçKoleksiyonu : IEnumerable<string> //IEnumerable Interface Realization
{
    private AraçTipiNumaralandırma enumerator;
    public AraçKoleksiyonu()
    {
        enumerator = new AraçTipiNumaralandırma();
    }
    public IEnumerator<string> GetEnumerator()
    {
        return enumerator;
    }
    IEnumerator IEnumerable.GetEnumerator() // Explicit implementation of
    → IEnumerable.GetEnumerator
    {
        return GetEnumerator();
    }
}

```

§14.2.7 yield Anahtar Kelimesi

Normal bir yöntemin aksine, içinde **yield** bulunan bir yöntem çalıştırıldığında tüm kod bloğu bir kerede icra edilmez. Bunun yerine yöntem, bir "durum makinesi" (**state machine**) gibi davranır ve her **yield return** satırına geldiğinde o anki değeri dışarıya fırlatıp, kaldığı yeri hatırlar.

Adım Adım Üretim

"Sonsuz Veri Akışları" veya "Büyük Veri Filtreleme" senaryosuyla geleneksel **List<T>** yönteminin neden yetersiz kaldığını ve **yield** anahtar kelimesinin belleği nasıl kurtardığını anlatacağız. Örnek senaryo olarak belleği yormadan milyonlarca iz kaydını (**log**) işlemeyi ele alalım. Elinizde milyonlarca satırlık bir iz kayıtları dosyası veya veri akışı olduğunu hayal edin. Amacınız sadece "HATA" (**error**) içeren kayıtları bulup ekrana basmak.

Geleneksel yöntemde, tüm hatalı kayıtları önce bir listeye doldurup sonra döndürürüz. Eğer 1 milyon hata varsa, 1 milyon nesne belleğe (RAM) hücum eder.

```

public List<string> HatalariGetir(List<string> tumLoglar)
{
    List<string> hatalar = new List<string>();
    foreach (var log in tumLoglar)
    {
        if (log.Contains("HATA"))
            hatalar.Add(log); // 1 milyon hata varsa RAM burada şişer!
    }
    return hatalar;
}

```

yield return yönteminde, bir liste oluşturulmaz. Program bir hata bulduğu anda onu dışarı fırlatır, işlenmesini bekler ve sonra kaldığı yerden aramaya devam eder. RAM'de sadece o anki tek bir satır tutulur. Bellek dostu olan bu yöntem **tembel değerlendirme (lazy evaluation)** olarak adlandırılır.

```

public IEnumerable<string> HatalariGetirYiseld(IEnumerable<string> tumLoglar)
{
    foreach (var log in tumLoglar)
    {
        if (log.Contains("HATA"))
        {
            // Buldum! Bunu al, işle, bitince geri gel.
            yield return log;
        }
    }
}

```

Bu örneği şu adımlarla açıklayabiliriz:

- ◇ Talep Anı: **foreach (var hata in HatalariGetirYield(loglar))** satırı çalıştığında yöntemin içine girilir.

- ◊ Bul ve Dur: İlk "HATA" satırı bulunduğunda `yield return` çalışır. Yöntem burada "donar" (durumunu kaydeder).
- ◊ Tüketim: Kontrol ana programa geçer, hata ekrana yazdırılır.
- ◊ Geri Dönüş: Döngü bir sonraki elemanı istediğinde, metot kaldığı yerden (dondduğu satırdan) devam eder; sanki hiç ayrılmamış gibi bir sonraki iz kaydını (`log`) kontrol eder.

yield return Kullanımı

`yield return` bir koleksiyondaki tüm elemanları bir kerede belleğe yüklemek yerine, her seferinde bir tanesini işlemek için kullanılır.

Bir başka örnek sonsuz sayı üreticini ele alabiliriz. C++'ta bunu bir `vector` ile yapmaya çalışmak saniyeler içinde `OutOfMemory` hatası verecektir.

```
public IEnumerable<int> SonsuzSayac()
{
    int i = 0;
    while (true) // Korkutucu görünüyor ama güvenli!
    {
        yield return i++;
    }
}

// Kullanımı:
foreach (var sayi in SonsuzSayac())
{
    Console.WriteLine(sayi);
    if (sayi > 100) break; // İstedığımız an dururuz, bellek tertemiz.
}
```

yield return ve İstisna İşleme

`yield return` kullanan bir yöntemin içinde `try-catch` bloğu kullanabilir misiniz? Peki `yield return` satırı bir `try` bloğu içindeyse `catch` ne zaman çalışır? `yield return` içeren bir yöntem içinde `try-catch` kullanabilirsiniz ancak bir kısıtlama vardır. `yield return` ifadesi içeren bir blokta `catch` bloğu bulunamaz (fakat `finally` bulunabilir). Çünkü yöntem donduğunda, dışarıdaki kodun ne zaman hata fırlatacağı belirsizdir. Hata yönetimi genellikle bu metodun çağırıldığı (tüketildiği) yerde yapılır.

Yinelemeyi Erken Sonlandırmak

Adım adım üretim (`yield return`) konusunu "üretim hattı" olarak anlattıktan sonra, `yield break` ifadesini bu hattın "Acil Durdurma Butonu" veya "Mantıksal Bitiş Noktası" olarak tanımlayabiliriz.

C++'ta bir fonksiyondan erken çıkmak için kullandığınız `return;` ifadesi neyse, bir yineleyici (`iterator`) metodu içinde de `yield break` odur. Ancak aralarında çok kritik bir fark vardır.

Bir `IEnumerable` metodu içinde döngü devam ediyor olsa bile, belirli bir koşul gerçekleştiğinde veri üretimi tamamen durdurmak ve koleksiyonun "sonuna gelindi" sinyalini vermek için `yield break` kullanılır.

- ◊ `yield break` satırı çalıştırıldığında: Yöntemin icrası o anda durur.
- ◊ Çağırın taraftaki `foreach` döngüsüne "artık veri kalmadı" (`MoveNext = false`) bilgisi gönderilir.
- ◊ Yöntem içindeki yerel değişkenler temizlenir ve kontrol çağrı yapan koda geri döner.

```
public IEnumerable<string> SansürlüMetinGetir(IEnumerable<string> kelimeler)
{
    foreach (var kelime in kelimeler)
    {
        if (kelime == "YASAKLI_İÇERİK")
        {
            // Bu kelimeyi gördüğün an yayını kes, sonrasını okuma bile!
            yield break;
        }

        yield return kelime;
    }
}
```

```

    }
    // Normal akış biterse burası çalışır ama yield break çalışırsa buraya ulaşamaz.
}

```

Bir `IEnumerable` metodu içinde sadece `return;` yazamazsınız; derleyici buna izin vermez. `return;` bildiğimiz yöntemlerde yöntemin icrasını sonlandırır ve varsa bir değer döndürür. Bu talimatın `yield` içeren yineleyici yöntemlerde doğrudan kullanımı yasaktır.

Bir veritabanından veri çekerken zaman aşımına uğradığınızda üretimi durdurmak için harikadır. Buna ilişkin kod örneği aşağıda verilmiştir;

```

public IEnumerable<Data> VeriSorgula(DateTime bitisZamani)
{
    while (true)
    {
        var veri = VeriTabanındanOku();

        if (DateTime.Now > bitisZamani)
        {
            Console.WriteLine("Zaman doldu, üretim durduruluyor...");
            yield break; // Döngü sonsuz olsa da burada kesilir.
        }

        yield return veri;
    }
}

```

Eğer `yield break` bir `try-finally` bloğu içindeyse, üretim durdurulmadan önce `finally` bloğu mutlaka çalıştırılır. Bu, C++'taki nesne yıkıcılarının (`destructor`) temizlik yapmasına benzer bir güvenlik sağlar.

içerik...

Neden yield Kullanmalıyız?

Geleneksel yöntem (`List` kullanmak) ile tüm liste belleğe yüklenir. Listenin tamamı dolana kadar bekler. Sonsuz döngü durumunda bellek dolar.

yield yönteminde (`IEnumerable`) sadece o anki eleman bellektedir (**lazy**). İlk eleman üretildiği an işlem başlar. Sonsuz sayı dizisi üretilebilir.

yield Niçin Kullanmalıyız

C# derleyicisi arka planda sizin için gizli bir sınıf oluşturur ve bu sınıf `IEnumerator` arayüzünü uygulayarak "geçerli eleman" (`Current`) ve "sonraki adıma geç" (`MoveNext`) mantığını yönetir. C++'ta bunu yapmak için manuel olarak yineleyici (`iterator`) sınıfları yazmanız gerekirdi.

Aşağıdaki kodu inceleyelim;

```

using System;
using System.Collections.Generic;
public class Program
{
    public static void Main(string[] args)
    {
        Console.WriteLine("1. Adım: Yöntem çağrılıyor...");
        // DİKKAT: Yöntem çağrılır ama içindeki kodlar HENÜZ çalışmaz!
        var sayiAkisi = TekSayilariGetir(5);

        Console.WriteLine("2. Adım: Yöntem çağrıldı, döngüye girmek üzereyiz.");
        Console.WriteLine("-----");

        // Döngü başladığı an Yöntem içindeki kodlar "talep üzerine" çalışmaya başlar.
        foreach (int sayi in sayiAkisi)
        {
            Console.WriteLine($"--> Ana Program: Yöntemtan {sayi} değeri geldi ve işleniyor.");
        }
        Console.WriteLine("-----");
    }
}

```



```

    Console.WriteLine("3. Adım: Tüm elemanlar işlendi, program bitti.");
}
public static IEnumerable<int> TekSayilariGetir(int limit)
{
    Console.WriteLine("    [Yöntem İçi]: Veri üretimi BAŞLADI.");
    for (int i = 1; i <= limit; i++)
    {
        if (i % 2 != 0)
        {
            Console.WriteLine($"    [Yöntem İçi]: {i} değeri bulundu, yield return ile
            ↳ gönderiliyor...");
            yield return i;
            Console.WriteLine("    [Yöntem İçi]: Ana program işini bitirdi, Yöntem kaldığı
            ↳ yerden devam ediyor.");
        }
    }
    Console.WriteLine("    [Yöntem İçi]: Döngü bitti, üretim TAMAMLANDI.");
}
}
/*Program Çıktısı:
1. Adım: Yöntem çağrılıyor...
2. Adım: Yöntem çağrıldı, döngüye girmek üzereyiz.
-----
[Yöntem İçi]: Veri üretimi BAŞLADI.
[Yöntem İçi]: 1 değeri bulundu, yield return ile gönderiliyor...
--> Ana Program: Yöntemtan 1 değeri geldi ve işleniyor.
[Yöntem İçi]: Ana program işini bitirdi, Yöntem kaldığı yerden devam ediyor.
[Yöntem İçi]: 3 değeri bulundu, yield return ile gönderiliyor...
--> Ana Program: Yöntemtan 3 değeri geldi ve işleniyor.
[Yöntem İçi]: Ana program işini bitirdi, Yöntem kaldığı yerden devam ediyor.
[Yöntem İçi]: 5 değeri bulundu, yield return ile gönderiliyor...
--> Ana Program: Yöntemtan 5 değeri geldi ve işleniyor.
[Yöntem İçi]: Ana program işini bitirdi, Yöntem kaldığı yerden devam ediyor.
[Yöntem İçi]: Döngü bitti, üretim TAMAMLANDI.
-----
3. Adım: Tüm elemanlar işlendi, program bitti.
*/

```

14.3 Koleksiyonlar ve Veri Yapıları

Koleksiyonlar (collections), verileri bellekte nasıl organize ettiğimizi belirler. C++ tarafındaki `std::vector`, `std::map` veya `std::list` yapılarına çok benzerler ancak C# dilinde bu yapılar `System.Collections.Generic` isim alanı altında çok daha optimize ve güvenli (type-safe) bir şekilde sunulur.

Doğru koleksiyonu seçmek, uygulamanızın hızı ve bellek kullanımı için hayattır.

§14.3.1 Dinamik Dizi List<T>

C++'taki `std::vector` yapısının tam karşılığıdır. Elemanlara indis ([0]) ile erişmek çok hızlıdır.

- ◊ **Özellik:** Boyutu dinamik olarak büyür. Sıralı bir yapıdır.
- ◊ **Performans:** Sona eleman eklemek $O(1)$, araya eleman eklemek veya silmek (kaydırma yaptığı için) $O(n)$ maliyetlidir.
- ◊ **Ne zaman kullanılır?** Eleman sayısı belirsiz olduğunda ve verilere indeksle erişmek gerektiğinde.

```

var liste = new List<int>() { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
liste.Add(6);
Console.WriteLine(liste.Count); // 6
liste.RemoveAt(3);
Console.WriteLine(liste.Count); // 5
Console.WriteLine(liste[3]); // 5

```

§14.3.2 Sözlük Dictionary<TKey, TValue>

C++'taki `std::unordered_map` yapısına benzer. Anahtar-değer çiftleri tutar.

- ◊ **Özellik:** Anahtarlar (key) benzersiz olmalıdır. Arka planda karma tablosu (hash table) kullanır.

- ◇ **Performans:** Bir anahtar üzerinden değere ulaşmak, eklemek ve silmek $O(1)$ maliyetlidir (inanılmaz hızlıdır).
- ◇ **Ne zaman kullanılır?** Bir kimlik (**identifier**) üzerinden veriye anında ulaşmak istendiğinde.

```
var kişiler = new Dictionary<int, string> { { 10, "İlhan" },
                                           { 20, "Mehmet" },
                                           { 30, "Abdullah" } };

Console.WriteLine(kişiler[10]); // "İlhan"
//Console.WriteLine(kişiler[50]); // HATA!:throws KeyNotFoundException
string? adı; //null olabilen string
if (kişiler.TryGetValue(20, out adı))
{
    Console.WriteLine(adı); // "Mehmet"
}
kişiler[40]="Abdullah";
kişiler.Add(50,"Damla");
//kişiler.Add(40, "Damla"); // HATA!:Throws ArgumentException since 40 already exists
foreach (KeyValuePair<int,string> kişi in kişiler)
{
    Console.WriteLine("No={0}, Adı={1}", kişi.Key, kişi.Value);
}
foreach (int no in kişiler.Keys)
{
    Console.WriteLine("No={0}", no);
}
foreach (string adı in kişiler.Values)
{
    Console.WriteLine("Adı={0}", adı);
}
```

§14.3.3 Benzersiz Küme HashSet<T>

Sadece benzersiz değerleri tutan, matematiksel küme mantığıyla çalışan bir koleksiyondur.

- ◇ **Özellik:** Aynı elemandan iki tane barındırmaz. Sıralama garantisi vermez.
- ◇ **Performans:** Eleman var mı yok mu kontrolü (**contains**) $O(1)$ maliyetlidir.
- ◇ **Ne zaman kullanılır?** Bir listede mükerrer kayıtları önlemek ve hızlı varlık kontrolü yapmak için.

```
HashSet<int> aidatVerenDaireler = new HashSet<int>();
aidatVerenDaireler.Add(4);
aidatVerenDaireler.Add(6);
aidatVerenDaireler.Add(9);
aidatVerenDaireler.Add(10);
Console.WriteLine($"Aidat Veren Daire Sayısı:{aidatVerenDaireler.Count}");
//Karma Küme Başlatma (İlk Değer Verme)
HashSet<int> aidatVermeyenDaireler = new HashSet<int>() { 1, 2, 3, 5, 8 };
bool vermemişmi = aidatVermeyenDaireler.Contains(8);
Console.WriteLine($"8 Numaralı Daire Aidat Vermemiş Mi:{vermemişmi}");
```

§14.3.4 Sıralı Küme SortedSet<T>

C++'taki `std::set` yapısına benzer. Elemanları her zaman sıralı tutar.

- ◇ **Özellik:** Kendi içinde elemanları otomatik sıralar.
- ◇ **Performans:** Ekleme ve arama işlemleri ikili arama ağacı (**binary search tree**) kullandığı için $O(\log n)$ maliyetlidir.
- ◇ **Ne zaman kullanılır?** Hem benzersizlik hem de sürekli sıralı bir liste gerektiğinde.

```
var sıralıListe = new SortedSet<int>();
sıralıListe.Add(4);
sıralıListe.Add(3);
sıralıListe.Add(3); // Kümede var eklenmez!
sıralıListe.Add(2);
sıralıListe.Add(1);
foreach (var item in sıralıListe)
{
    Console.Write(item+" ");
}
```

```

}
/*Program Çıktısı:
1 2 3 4
*/

```

§14.3.5 Yığın Stack<T>

"Son gelen, ilk çıkar" LIFO (**last in first out**) prensibiyle çalışır. Bir kutuya kitapları üst üste dizmek gibidir.

- ◊ **Özellik:** Sadece en üstteki elemana erişebilirsiniz (**peek** veya **pop**).
- ◊ **Ne zaman kullanılır?** Geriye dönük işlemler (**undo/redo**), fonksiyon çağrılarını (**stack unwinding**) gibi senaryolarda.

```

var yığın = new Stack<int>();
yığın.Push(10);
yığın.Push(3);
yığın.Push(5);
yığın.Push(8);
Console.WriteLine(yığın.Peek()); // 8
Console.WriteLine(yığın.Pop()); // 8
Console.WriteLine(yığın.Pop()); // 5
Console.WriteLine(yığın.Pop()); // 3

```

§14.3.6 Kuyruk Queue<T>

"İlk gelen, ilk çıkar" FIFO (**first in first out**) prensibiyle çalışır. Ekmek sırasına girmek gibidir.

- ◊ **Özellik:** Elemanlar arkadan eklenir (**enqueue**), önden çıkarılır (**dequeue**).
- ◊ **Ne zaman kullanılır?** İşlem sırasına alınmış görevler, yazıcı kuyrukları veya mesaj sistemlerinde.

```

var kuyruk = new Queue<int>();
kuyruk.Enqueue(10);
kuyruk.Enqueue(6);
kuyruk.Enqueue(4);
kuyruk.Enqueue(9);
Console.WriteLine(kuyruk.Peek()); // 10
Console.WriteLine(kuyruk.Dequeue()); // 10
Console.WriteLine(kuyruk.Dequeue()); // 6
Console.WriteLine(kuyruk.Dequeue()); // 4

```

§14.3.7 Bağlı Liste LinkedList<T>

C++'taki `std::list` yapısıdır. Elemanlar bellekte ardışık değil, birbirlerine referanslarla (göstericilerle) bağlıdır.

- ◊ **Özellik:** Çift yönlüdür (**doubly linked list**).
- ◊ **Performans:** Aradan eleman silmek veya eklemek (sadece referans güncellendiği için) çok hızlıdır. Ancak indeksle erişim yoktur, eleman bulmak için listeyi taramak gerekir.
- ◊ **Ne zaman kullanılır?** Listenin ortasına çok sık ekleme-çıkarma yapılıyorsa.

```

LinkedList<int> bağlıListe = new LinkedList<int>();
bağlıListe.AddLast(3);
bağlıListe.AddLast(5);
bağlıListe.AddLast(8);
bağlıListe.AddFirst(10);
bağlıListe.AddFirst(2);
bağlıListe.RemoveFirst();
bağlıListe.RemoveLast();
foreach (int i in bağlıListe)
Console.Write(i+" ");
/* Program Çıktısı:
10 3 5
*/

```

§14.3.8 Özet Tablo

14.4 Performans ve Bellek Optimizasyon Jenerikleri

Bu yapılar, "bir nesneye gerçekten ihtiyaç duyulana kadar onu bellekte oluşturma" prensibine dayanır.

Koleksiyon	Erişim Hızı	Ekleme/Silme	Sıralama	Benzersizlik
List	$O(1)$ (İndeks)	$O(n)$	Eklendiği sıra	Hayır
Dictionary	$O(1)$ (Key)	$O(1)$	Belirsiz	Key: Evet
HashSet	$O(1)$ (Var mı?)	$O(1)$	Belirsiz	Evet
SortedSet	$O(\log n)$	$O(\log n)$	Sıralı	Evet
LinkedList	$O(n)$	$O(1)$	Eklendiği sıra	Hayır

Tablo 14.2: Koleksiyonlar Özet Tablosu

§14.4.1 Lazy<T>

En yaygın kullanılan yapıdır. **Tembel başlatma** (**lazy initialization**), nesnenin oluşturulma maliyeti yüksekse ve her senaryoda kullanılmayacaksa tercih edilir. İlk erişimde (**Value** özelliği çağrıldığında) nesneyi oluşturur ve sonraki çağrılarda aynı örneği döner. Varsayılan olarak iş parçacığı güvenlidir (**thread-safe**); yani birden fazla iş parçacığı (**thread**) aynı anda erişse bile nesne sadece bir kez güvenli bir şekilde oluşturulur.

Düşünelim ki çok ağır bir veritabanı yükleme işleminiz var. Bu nesneyi program açılır açılmaz oluşturmak yerine, ilk defa bir veri çekilmek istendiğinde oluşturulmasını sağlayalım.

```
using System;
public class AgirVeritabaniBaglantisi
{
    public AgirVeritabaniBaglantisi()
    {
        Console.WriteLine(">>> Çok ağır veritabanı bağlantısı kuruluyor... (5 saniye sürdü)");
        // Burada ağır bir yükleme işlemi olduğunu varsayalım.
    }
    public void VeriGetir() => Console.WriteLine("Veriler başarıyla getirildi.");
}
class Program
{
    static void Main()
    {
        // 1. Tanımlama: Nesne henüz oluşturulmadı.
        // Sadece nasıl oluşturulacağını (factory method) söyledik.
        Lazy<AgirVeritabaniBaglantisi> lazyBaglanti = new Lazy<AgirVeritabaniBaglantisi>(() =>
            new AgirVeritabaniBaglantisi());
        Console.WriteLine("Program başladı. Nesne henüz bellekte yok.");
        Console.WriteLine("-----");
        // 2. İlk Erişim: Value özelliği çağrıldığı anda constructor çalışır.
        Console.WriteLine("Veritabanına şimdi ihtiyaç duyuluyor...");
        lazyBaglanti.Value.VerGetir();
        // 3. İkinci Erişim: Nesne zaten olduğu için constructor tekrar çalışmaz.
        Console.WriteLine("\nİkinci defa veri isteniyor...");
        lazyBaglanti.Value.VerGetir();
        // Nesnenin oluşup oluşmadığını kontrol edebiliriz:
        Console.WriteLine($"Nesne oluştu mu? : {lazyBaglanti.IsValueCreated}");
    }
}
```

Varsayılan olarak **Lazy<T>** nesnesi, birden fazla iş parçacığı (**thread**) aynı anda **Value** özelliğine erişse bile nesnenin sadece bir kez oluşturulacağını garanti eder (**race condition**'ı önler).

§14.4.2 LazyInitializer

Lazy<T> sınıfının sağladığı konforu, ek bir nesne sarmalayıcı (**wrapper**) oluşturmadan, doğrudan mevcut değişkenler üzerinden sağlar. Bellek kullanımının çok kritik olduğu, **Lazy<T>** nesnesinin kendisinin yaratacağı küçük ek yükün bile istenmediği durumlarda (örneğin binlerce nesne içeren listeler) kullanılır.

EnsureInitialized yöntemi ile çalışır.

Lazy<T> kullandığınızda, asıl nesnenizden önce bir de **Lazy** nesnesi oluşturursunuz. Eğer sisteminizde milyonlarca "belki oluşturulacak" nesne varsa, **Lazy<T>**'nin kendisi ciddi bir bellek yükü oluşturabilir.

LazyInitializer ise hiç ek nesne oluşturmadan, doğrudan **ref** anahtar kelimesiyle mevcut değişkeniniz üzerinde çalışır.

```
using System;
```

```

using System.Threading;
public class AgirRapor
{
    public AgirRapor() => Console.WriteLine(">>> Ağır Rapor Nesnesi Oluşturuldu.");
    public void Yazdir() => Console.WriteLine("Rapor içeriği basılıyor...");
}
class Program
{
    // 1. Ham değişkenimizi tanımlıyoruz (Henüz null)
    private static AgirRapor _rapor;
    static void Main()
    {
        Console.WriteLine("Program Başladı...");
        // 2. EnsureInitialized: Değişken null ise oluşturur, değilse mevcut olanı döner.
        // Thread-safe (iş parçacığı güvenli) bir şekilde çalışır.
        LazyInitializer.EnsureInitialized(ref _rapor, () =>
        {
            return new AgirRapor();
        });
        _rapor.Yazdir();
        // Tekrar çağrıldığında constructor çalışmaz, mevcut olan kullanılır.
        LazyInitializer.EnsureInitialized(ref _rapor);
        _rapor.Yazdir();
    }
}

```

§14.4.3 İş Parçacığı Yönetimi ve İzole Veri Yapıları

Modern yazılım geliştirme süreçlerinde, uygulamaların aynı anda birden fazla işi yapabilmesi (**concurrency**) vazgeçilmez bir gereksinimdir. C#, bu süreci yönetmek için düşük seviyeli **Thread** sınıflarından, yüksek seviyeli **Task** tabanlı asenkron yapılara kadar geniş bir yelpaze sunar.

Thread, işletim sistemi seviyesindeki en küçük yürütme birimidir. Gerçek bir işçidir. Bir **Thread** başlattığınızda, işletim sisteminden kaynak yani bellek, yığın (**stack**) alanı talep edersiniz. Her **Thread** yaklaşık 1 MB yığın alanı kaplar. Binlerce iş parçacığı oluşturmak sistemi kilitleyebilir.

async bir anahtar kelimedir ve bir yöntemin "beklenebilir" (**awaitable**) olduğunu, yani bu metodun içinde zaman alan bir işlem yapılabileceğini belirtir. Bir yöntemin imzasını değiştirir. Bu yöntemin mutlaka bir **Task** veya **Task<T>** döndüreceğini taahhüt eder. **async** tek başına bir işi arka planda çalıştırmaz; sadece o yöntemin içinde **await** kullanılmasına olanak tanır.

await, asenkron bir işlem bitene kadar metodun yürütülmesini askıya alır (**suspend**) ancak bu sırada ana iş parçacığını bloklamaz. "Bu işlem bitene kadar buradaki işi durdur, ama iş parçacığı boşa çıksın ve başka işlere baksın. İşlem bittiğinde kaldığın yerden devam et" demektir.

Özellik	Thread	Task (async/await)
Seviye	Düşük seviyeli işletim sistemi birimi.	Yüksek seviyeli soyutlama (promise).
Kaynak	Ağır (Her biri 1MB Stack).	Hafif (Sadece bir nesne referansı).
Kullanım	CPU yoğunluklu işler (Matematiksel hesaplama).	I/O yoğunluklu işler (Dosya, Ağ, Veritabanı).
Bloklama	Thread'i durdurur.	Thread'i serbest bırakır.

Tablo 14.3: Thread ve Task Karşılaştırması

Bir restoranda olduğunuzu hayal edin:

- ◇ **Thread** (Senkron): Garson siparişi açığıya verir ve yemek pişene kadar mutfağın önünde bekler. Başka hiçbir masaya bakamaz. (Bloklama).

```
public void VeriIsle() //Bloklayan Yaklaşım!
```

```

{
    Thread.Sleep(5000); // Thread burada 5 saniye boyunca donar. UI cevap vermez.
    Console.WriteLine("İşlem bitti");
}

```

- ◇ **Çoklu İş Parçacığı (multi-threading)**: Her masa için yeni bir garson işe alırsınız. Yemekler pişerken

hepsi bekler. Çok fazla masa gelirse dükkân garsonla dolar, batarsınız. (Kaynak tüketimi).

- ♦ `async/await`: Garson siparişi verir, mutfağa "Hazır olunca zile bas" der ve gidip diğer masaların siparişini alır. Zil çalınca (işlem bitince) geri döner ve yemeği servis eder. (Verimlilik).

```
public async Task VeriIsleAsync() //Verimli Yaklaşım.
```

```
{
```

```
    await Task.Delay(5000); // Thread boşa çıkar, başka işler yapar. // 5 saniye sonra
```

```
    → sistem kaldığı yerden devam eder.
```

```
    Console.WriteLine("İşlem bitti");
```

```
}
```

İş Parçacığı İzolasyonunun Önemi

Çok iş parçacıklı (**multi-threaded**) ortamlarda en büyük zorluk, paylaşılan verilerin (**shared state**) güvenli bir şekilde yönetilmesidir. Kilitlenme (**locking**) mekanizmaları veri güvenliğini sağlasa da, performans darboğazlarına neden olabilir. Bu noktada, veriyi iş parçacığına veya program akışına özel hale getiren izolasyon yapıları devreye girer.

ThreadLocal<T>

Verinin her iş parçacığı (**thread**) için özel bir kopyasını tutar. Bir değişkenin değeri "Thread A" için farklı, "Thread B" için farklıdır. İş parçacıkları arası kilitlenmeleri (**locking**) önlediği için paralel programlamada performansı artırır.

Diyelim ki bir döngü içinde birden fazla iş parçacığı çalıştırıyoruz ve her iş parçacığının kendine has bir "sayaç" değerine sahip olmasını istiyoruz.

```
using System;
using System.Threading;
using System.Threading.Tasks;
class Program
{
    // Her thread için özel bir tam sayı deposu oluşturuyoruz. Başlangıç değeri olarak her
    → thread'e rastgele bir ID veriyoruz.
    private static ThreadLocal<int> _threadOzelDeger = new ThreadLocal<int>(() =>
    {
        return Thread.CurrentThread.ManagedThreadId * 100;
    });
    static void Main()
    {
        // 3 farklı işi paralel olarak başlatalım
        Parallel.For(0, 3, i =>
        {
            // Her thread kendi özel kopyasını artırır
            _threadOzelDeger.Value++;
            Console.WriteLine($"Thread ID:
            → {Thread.CurrentThread.ManagedThreadId}, Özel Değeri:
            → {_threadOzelDeger.Value}");
        });
        // Main thread'in kendi kopyası bunlardan etkilenmez
        Console.WriteLine($"Main Thread Değeri: {_threadOzelDeger.Value}");
        // Kaynakları temizle
        _threadOzelDeger.Dispose();
    }
}
```

AsyncLocal<T>

Modern asenkron yöntemlerde (`async/await`), bir işlem başladıktan sonra `await` satırında farklı bir iş parçacığına (**thread**) geçebilir. `AsyncLocal<T>`, verinin iş parçacığından bağımsız olarak mantıksal işlem akışını (**logical control flow**) takip etmesini sağlar.

`ThreadLocal<T>`'den farklı olarak, verinin asenkron yöntemlerde (`async/await`), (`await`) arasında kaybolmadan taşınmasını sağlar. Örneğin, bir kullanıcının "Kültür/Dil" bilgisini tüm asenkron çağrılar boyunca taşımak için kullanılır.

Düşünelim ki bir web isteği aldınız ve bu isteğe özel bir `KullanıcıID` değerini, metotlara parametre olarak tek tek geçmeden en alt katmanlara kadar ulaştırmak istiyorsunuz.

```
using System;
using System.Threading;
using System.Threading.Tasks;
class Program
{
    // Akış boyunca taşınacak asenkron yerel değişken
    private static AsyncLocal<string> _kullaniciKodu = new AsyncLocal<string>();
    static async Task Main()
    {
        // 1. Akış: Ahmet için başlatılıyor
        await İşlemBaşlat("Ahmet");
        // 2. Akış: Mehmet için başlatılıyor
        await İşlemBaşlat("Mehmet");
    }
    static async Task İşlemBaşlat(string isim)
    {
        _kullaniciKodu.Value = isim;
        Console.WriteLine($"[Baslangic] Akis sahibi: {_kullaniciKodu.Value}, Thread ID:
        ↳ {Thread.CurrentThread.ManagedThreadId}");
        await AltYöntem();
    }
    static async Task AltYöntem()
    {
        // Bir asenkron bekleme yapıyoruz. Bu noktadan sonra farklı bir thread'e geçebiliriz.
        await Task.Delay(100);
        // AsyncLocal sayesinde veri hala burada!
        Console.WriteLine($"[Alt Yöntem] Akis sahibi: {_kullaniciKodu.Value}, Thread ID:
        ↳ {Thread.CurrentThread.ManagedThreadId}");
        await EnAltKatman();
    }
    static async Task EnAltKatman()
    {
        await Task.Delay(50);
        Console.WriteLine($"[En Alt] Akis sahibi: {_kullaniciKodu.Value}, Thread ID:
        ↳ {Thread.CurrentThread.ManagedThreadId}");
    }
}
```

Veri İzolasyon Yapıları Karşılaştırması

Aşağıdaki tablo, C# içerisinde verinin nasıl izole edilebileceğini ve asenkron akışlarla olan uyumunu özetlemektedir:

Yapı	Kapsam / İzolasyon Seviyesi	async/await Desteği
<code>static</code>	Uygulama Geneli (Paylaşılan bellek)	İzolasyon Yok
<code>ThreadLocal<T></code>	Fiziksel İş Parçacığı (thread-bound)	Yok (<code>await</code> sonrası kaybolur)
<code>AsyncLocal<T></code>	Mantıksal Akış (execution context)	Tam Destek (Akışı takip eder)

Tablo 14.4: Veri Saklama ve Taşıma Yapıları Karşılaştırması

Asenkron Hata Ayıklama

Asenkron programlamada hata yönetimi, senkron koddan farklı olarak istisnanın (**exception**) fırlatıldığı iş parçacığı ile yakalandığı iş parçacığının aynı olmaması nedeniyle özel bir dikkat gerektirir.

async void Tehlikesi : Asenkron hata yönetiminde en büyük düşman `async void` kullanımıdır. `async void` ile işaretlenmiş bir yöntem hata fırlattığında, çağırıcı (**caller**) bu hatayı `try-catch` ile yakalayamaz. Hata doğrudan uygulamanın işlem (**process**) seviyesine çıkar ve genellikle uygulamanın çökmesine neden olur. Olay işleyiciler (**event handlers**) dışında her zaman `async Task` kullanın. `Task` döndüren yöntemlerde istisna, dönen `Task` nesnesinin içine paketlenir.

try-catch Kullanımı: Asenkron metotlarda `try-catch` blokları senkron koda çok benzer şekilde çalışır,

ancak arka planda derleyici (**state machine**) hatayı sizin için doğru bağlama taşır.

```
public async Task VeriIsleAsync()
{
    try
    {
        await ServisCagrisi();
    }
    catch (HttpRequestException ex)
    {
        // Hata burada yakalanır, await sayesinde doğru context'e (bağlama) geri dönmüştür.
        Console.WriteLine($"Hata oluştu: {ex.Message}");
    }
}
```

Birden Fazla Hata (**AggregateException**): Aynı anda birden fazla asenkron işi başlattığınızda (**Task.WhenAll**), birden fazla hata oluşabilir. **await** Davranışı: **await Task.WhenAll(...)** kullandığınızda, yakalanan istisna sadece ilk oluşan hatadır. Tüm Hataları Yakalamak: Eğer tüm hataları görmek istiyorsanız, **Task** nesnesinin **Exception** özelliğini incelemeniz gerekir.

```
Task tumIsler = Task.WhenAll(task1, task2, task3);
try
{
    await tumIsler;
}
catch (Exception)
{
    // Sadece ilk hata burada yakalanır. Tüm hatalara erişmek için:
    foreach (var ex in tumIsler.Exception.InnerExceptions)
    {
        Console.WriteLine(ex.Message);
    }
}
```

ConfigureAwait(false) ve Hatalar: Kütüphane (**library**) geliştirirken sıkça kullanılan **ConfigureAwait(false)**, hatanın yakalandığı iş parçacığını değiştirebilir. Eğer hatayı yakaladıktan sonra bir UI bileşenine (örneğin **WinForms Label**) erişecekseniz, bu durum "**Cross-thread operation not valid**" hatasına yol açar.

Senaryo	Önerilen Yöntem	Risk / Dikkat
async Task	Standart try-catch .	İstisna Task içine paketlenir.
async void	Sadece Event Handler 'larda kullanın.	Hata yakalanamaz, uygulama çökebilir.
Task.WhenAll .Result / .Wait()	AggregateException kontrolü. Kullanmaktan kaçının.	await sadece ilk hatayı döner. AggregateException fırlatır ve kilitlenme (deadlock) riski taşır.

Tablo 14.5: Asenkron Hata Yönetimi Yöntemleri

§14.4.4 Performans Notları

Eğer bellek kullanımı kritikse **LazyInitializer** statik sınıfı tercih edilmelidir. **Lazy<T>** nesnesinin oluşturacağı ek yükten kaçınılmak isteniyorsa da bu yapı kullanılır. Bu yapı, ham göstericiler (**raw pointers**) üzerinde **atomic** operasyonlar yürüterek düşük seviyeli ve yüksek performanslı bir başlatma sağlar.

- ♦ **Lazy<T>**: Nesne tabanlı, kolay kullanım, iş parçacığı güvenlidir (**thread-safe**).
- ♦ **LazyInitializer**: Statik yardımcı, düşük bellek tahsisi, yüksek performans.

ThreadLocal Kullanımı

İş parçacığı yönetiminde uygulamanın zaman uyumsuz (**asynchron**) olup olmadığına dikkat edilmelidir. Eğer **Task** ve **await** yapıları yoğun kullanılıyorsa **AsyncLocal<T>**, düşük seviyeli iş parçacığı havuzları yönetiliyorsa **ThreadLocal<T>** doğru tercihtir.

14.5 İleri Seviye Jenerik Mimariler

Bu yapılar, kodun tip güvenliğini (**type safety**) sağlarken performansı maksimize eder.

Yapı	Temel Odak	Kritik Avantaj
<code>Lazy<T></code>	Nesne Ömrü Yönetimi	Kolay kullanım, otomatik iş parçacığı-güvenli (thread-safety).
<code>LazyInitializer</code>	Düşük Bellek Tüketimi	Ekstra nesne yaratmadan başlatma yapar.
<code>ThreadLocal<T></code>	Paralel Programlama	İş parçacıkları arası izole veri saklar.
<code>AsyncLocal<T></code>	Asenkron Akış	<code>async/await</code> akışında veriyi taşır.

Tablo 14.6: Ertelenmiş Başlatma ve Veri Saklama Yapıları

§14.5.1 Nullable<T>

Sadece değer tipleri için kullanılır. Aslında `int?` veya `bool?` olarak kullandığımız yapıların arkasındaki jenerik `struct` yapısıdır. Yığında olan değer tiplerinin (**stack-based**) `null` değer alabilmesini sağlar. Veritabanı işlemlerinde (opsiyonel alanlar) hayati öneme sahiptir. `HasValue` ve `Value` özellikleriyle kontrol edilir. 4.7 başlığında anlatılmıştı.

§14.5.2 ValueTuple<T1, T2, ...>

C#7.0+ ile gelmiştir. Yöntemlerden birden fazla değer döndürmek için kullanılan hafif (**lightweight**) bir yapıdır. `Tuple<T>` bir sınıftır ve öbek bellekte (**heap**) bulunur, `ValueTuple` bir yapıdır ve yığın bellekte (**stack**) bulunur. Bu da onu çok daha performanslı kılar. İsimlendirilebilir alanlar (**field**) sunar.

Aşağıdaki örnekte, bir dizideki hem en küçük hem de en büyük değeri aynı anda döndüren bir metod tasarlayalım. Eskiden bunun için `out` parametreleri veya özel bir `class` kullanmamız gerekirdi.

```
using System;
using System.Linq;
public class IstatistikAraci
{
    // Yöntemin dönüş tipi (int Min, int Max) olan bir ValueTuple'dır.
    public (int Min, int Max) Hesapla(int[] sayilar)
    {
        if (sayilar == null || sayilar.Length == 0)
            return (0, 0);
        int min = sayilar.Min();
        int max = sayilar.Max();
        // Geriye isimlendirilmiş bir ValueTuple döndürüyoruz.
        return (min, max);
    }
}
class Program
{
    static void Main()
    {
        var arac = new IstatistikAraci();
        int[] veriler = { 45, 12, 89, 3, 67 };
        // 1. Kullanım: Doğrudan isimlendirilmiş alanlara erişim
        var sonuc = arac.Hesapla(veriler);
        Console.WriteLine($"En Küçük: {sonuc.Min}, En Büyük: {sonuc.Max}");
        // 2. Kullanım: Deconstruction (Nesneyi parçalarına ayırma)
        var (enKucuk, enBuyuk) = arac.Hesapla(veriler);
        Console.WriteLine($"Parçalanmış Veri: {enKucuk} ve {enBuyuk}");
    }
}
```

Özellik	Tuple (Eski)	ValueTuple (Yeni)
Tip	Reference Type (<code>class</code>)	Value Type (<code>struct</code>)
Bellek	Öbek (heap) bellekte yer alır.	Yığın (stack) üzerinde tutulur.
Sorumluluk	<code>Item1</code> , <code>Item2</code> (Sabit isimler)	<code>Min</code> , <code>Max</code> (Özel isimler)
Performans	Daha yavaştır (çöp toplayıcı baskısı).	Çok hızlıdır.

Tablo 14.7: ValueTuple Teknik Karşılaştırması

`ValueTuple` kullanarak;

- ◊ Dönüş tipindeki alanlara `Isim`, `Yaş` gibi anlamlı isimler verebilirsiniz. `Tuple` kullanarak eskiden sadece

`Item1`, `Item2` diyebiliyordunuz.

- ◊ Bir `struct` olduğu için öbek (**heap**) yerine yığın (**stack**) belleği kullanır, bu da çöp toplayıcı (GC) üzerindeki yükü azaltır.
- ◊ `var (x, y) = GetPoint();` şeklinde tek satırda parçalarına ayırabilirsiniz.

§14.5.3 EqualityComparer<T>

Jenerik koleksiyonlarda (`Dictionary` veya `HashSet` gibi) nesnelerin birbirine eşit olup olmadığını kontrol etmek için kullanılan standart yapıdır. `T` tipinin `Equals()` yöntemini her seferinde çağırmak yerine, tipin en uygun karşılaştırma yöntemini (`IEquatable<T>` gibi) otomatik olarak seçer. Kutulama (**boxing/unboxing**) işlemlerini önleyerek büyük veri setlerinde hız kazandırır.

Diyelim ki elimizde `Urun` nesneleri var ve biz bu nesneleri bir `HashSet` içinde saklamak istiyoruz. Varsayılan olarak `C#`, nesnelerin bellek adreslerine bakar. Ancak biz, Ürün Kodu aynı olan ürünlerin "aynı" sayılmasını istiyorsak kendi karşılaştırmamızı yazmalıyız.

```
using System;
using System.Collections.Generic;
public class Urun
{
    public string UrunKodu { get; set; }
    public string Ad { get; set; }
}
// 1. Karşılaştırmacı Sınıf: EqualityComparer<T>'den türetilir.
public class UrunKarsilastirici : EqualityComparer<Urun>
{
    public override bool Equals(Urun x, Urun y)
    {
        // Null kontrolleri
        if (ReferenceEquals(x, y)) return true;
        if (x == null || y == null) return false;
        // Ürün koduna göre eşitlik kontrolü
        return x.UrunKodu == y.UrunKodu;
    }
    public override int GetHashCode(Urun obj)
    {
        if (obj == null) return 0;
        // Hash kodu olarak ürün kodunun hash'ini döndürürüz
        return obj.UrunKodu.GetHashCode();
    }
}
class Program
{
    static void Main()
    {
        // Karşılaştırmacıyı koleksiyona enjekte ediyoruz
        HashSet<Urun> urunler = new HashSet<Urun>(new UrunKarsilastirici());
        Urun u1 = new Urun { UrunKodu = "ELMA1", Ad = "Amasya Elması" };
        Urun u2 = new Urun { UrunKodu = "ELMA1", Ad = "Yeşil Elma" }; // Kod aynı!
        urunler.Add(u1);
        urunler.Add(u2); // Bu eklenmeyecektir çünkü kodlar aynı sayılır.
        Console.WriteLine($"Listedeki Ürün Sayısı: {urunler.Count}"); // Çıktı: 1
    }
}
```

§14.5.4 IProgress<T>

Zaman uyumsuz (**asynchron**) işlemlerde (özellikle uzun süren döngülerde) ilerleme bilgisini UI katmanına veya çağıran metoda güvenli bir şekilde iletmek için kullanılır. `Task.Run` içinde çalışırken `Report(T value)` yöntemi ile ilerleme yüzdesi veya durumu gönderilir. `C#`'ın `async/await` mimarisiyle tam uyumludur.

`IProgress<T>`, özellikle uzun süren bir işlem (dosya indirme, veritabanı işleme vb.) sırasında kullanıcı arayüzüne veya kullanıcıya "işlemin % kaç bitti" bilgisini asenkron bir şekilde iletmek için kullanılır. Diyelim ki 1'den 10'a kadar sayan ve her adımda ilerlemeyi raporlayan asenkron bir yöntemimiz var.

```
using System;
```

```

using System.Threading.Tasks;
public class İşlemci
{
    // IProgress<int> parametresi, ilerleme bilgisini taşıyan kanaldır.
    public async Task UzunSürenİşlem(IProgress<int> progress)
    {
        for (int i = 1; i <= 10; i++)
        {
            await Task.Delay(500); // Gerçek bir iş yapılmış gibi bekleme
            progress?.Report(i * 10); // İlerlemeyi raporla (% cinsinden veya adım sayısı)
        }
    }
}
class Program
{
    static async Task Main()
    {
        İşlemci işlemci = new İşlemci();
        // 1. Progress nesnesi oluşturulur ve rapor geldiğinde ne yapılacağı tanımlanır.
        var progressKanalı = new Progress<int>(yuzde =>
        {
            Console.WriteLine($"İlerleme: %{yuzde}%");
        });
        Console.WriteLine("İşlem başlatılıyor...");
        // 2. Yöntem çağrılırken bu kanal parametre olarak geçilir.
        await işlemci.UzunSürenİşlem(progressKanalı);
        Console.WriteLine("İşlem tamamlandı!");
    }
}

```

§14.5.5 Comparer<T> ve IComparable<T>

Sıralama (**sorting**) algoritmalarının temelini oluşturan jenerik yapılardır. İki nesne arasındaki büyük-lük/küçüklük ilişkisini belirler. **Default** özelliği ile o tip için tanımlanmış varsayılan sıralama kuralına erişilir.

IComparable<T>, bir sınıfın kendi içindeki varsayılan sıralama kuralıdır. Örneğin bir **Öğrenci** listesini her zaman "Not" değerine göre sıralamak istiyorsanız bu arayüzü kullanırsınız.

```

using System;
using System.Collections.Generic;
public class Öğrenci : IComparable<Öğrenci>
{
    public string Ad { get; set; }
    public int Not { get; set; }
    // Kendimi (this) diğeriyle (other) kıyaslıyorum
    public int CompareTo(Öğrenci other)
    {
        if (other == null) return 1;
        // Not değerine göre küçükten büyüğe sıralama
        return this.Not.CompareTo(other.Not);
    }
}

```

Comparer<T>, eğer varsayılan kuralın dışında (örneğin bazen isme göre, bazen nota göre) sıralama yapmak istiyorsanız, dışarıdan bir karşılaştırıcı enjekte edersiniz.

```

public class ÖğrenciAdınaGöreSırala : Comparer<Öğrenci>
{
    public override int Compare(Öğrenci x, Öğrenci y)
    {
        // Null kontrolleri
        if (x == null && y == null) return 0;
        if (x == null) return -1;
        if (y == null) return 1;
        // İsme göre alfabetik sıralama
    }
}

```

```

        return string.Compare(x.Ad, y.Ad);
    }
}

```

§14.5.6 Özet

Yapı	Veri Tipi	Bellek	Temel Kullanım Senaryosu
Nullable<T>	struct	Yığın (stack)	Opsiyonel sayısal/mantıksal veriler.
ValueTuple	struct	Yığın (stack)	Hızlı ve geçici çoklu veri döndürme.
Lazy<T>	class	Öbek (heap)	Maliyetli nesnelerin ertelenmiş başlatılması.
EqualityComparer	class	Öbek (heap)	Koleksiyonlarda nesne eşleşme mantığı.

Tablo 14.8: Kritik Jenerik Yapıların Teknik Analizi

14.6 İndisleyiciler

İndisleyiciler (indexer), nesneleri bir dizi gibi kullanmayı sağlarlar. İndisleyiciler, köşeli parantez içinde parametresi olan **this** kelimesiyle tanımlanırlar. Özellikler (**property**) gibi indisleyiciler de **get** ve **set** yöntemleri kullanılır ve yalnız okunur (**read only**), yalnız yazılır (**write only**) olabilirler. Bunların dışında indisleyiciler sanal olarak da tanımlanabilir ve parametresi farklı olmak kaydıyla indisleyicilere aşırı yükleme (**overload**) yapılabilir.

```

İndisliListe<string> names = new İndisliListe<string>();
names.Add("Mehmet");
names.Add("Abdullah");
names.Add("İlhan");
for (int i = 0; i < names.ElemanSayısı; i++)
{
    string s = names[i]; //indis kullanımı:
    names[i] = s.ToUpper();
    Console.WriteLine(names[i]);
}

public class İndisliListe<T>
{
    //Sabitler:
    const int varsayılanElemanKapasitesi = 4;
    //Alanlar:
    T[] elemanlar;
    int elemanSayısı;
    //Yapıcılar:
    public İndisliListe(int kapasite)
    {
        elemanlar = new T[kapasite];
    }
    public İndisliListe() : this(varsayılanElemanKapasitesi) { }

    //Özellikler:
    public int ElemanSayısı
    {
        get { return elemanSayısı; }
    }
    public int Kapasite
    {
        get { return elemanlar.Length; }
        set
        {
            if (value < elemanSayısı) value = elemanSayısı;
            if (value != elemanlar.Length)
            {
                T[] yeniElemanlar = new T[value];
                Array.Copy(elemanlar, 0, yeniElemanlar, 0, elemanSayısı);
                elemanlar = yeniElemanlar;
            }
        }
    }
}

```

```

    }
}
//İndisleyici Tanımı:
public T this[int indis]
{
    get { return elemanlar[indis]; }
    set
    {
        if (indis < 0 || indis >= elemanSayısı)
            throw new ArgumentOutOfRangeException("İndis Hatası");
        elemanlar[indis] = value;
    }
}
//Yöntemler:
public void Add(T eleman)
{
    if (elemanSayısı == Kapasite) Kapasite = elemanSayısı * 2;
    elemanlar[elemanSayısı] = eleman;
    elemanSayısı++;
}
}

```

14.7 Düşün ve Çöz

- ◊ **Düşün:** Jenerik programlama neden tip güvenliği ve performans açısından jenerik olmayan (**object** tabanlı) koleksiyonlara üstündür? Eski **ArrayList** ile **List<T>** arasındaki farkı kutulama/kutudan çıkarma üzerinden açıklayınız.
- ◊ **Düşün:** Jenerik kısıtlamalar (**constraints: where T : class, where T : new(), where T : IComparable**) ne amaçla kullanılır? Kısıtlamalar olmadan jenerik bir algoritmanın hangi işlemleri gerçekleştiremeyeceğini örnekleyiniz.
- ◊ **Düşün:** **List<T>**, **Dictionary<K,V>**, **HashSet<T>**, **Stack<T>** ve **Queue<T>** koleksiyonlarının hangi senaryolarda tercih edildiğini karşılaştırınız. Arama (**lookup**) hızı açısından neden **Dictionary** ve **HashSet** öne çıkar?
- ◊ **Düşün:** **ConcurrentDictionary<K,V>** ve diğer thread-safe koleksiyonlar ne zaman gereklidir? Normal **Dictionary** üzerinde **lock** kullanmakla kıyaslandığında performans ve kullanım kolaylığı açısından farkı nedir?
- ◊ **Düşün:** İndisleyici (**indexer**) özelliği nedir? Sınıfınızı dizi gibi kullanılabilir kılmanın avantajı ne olabilir? Çok boyutlu indisleyici tanımlamak mümkün müdür?
- ◊ **Düşün:** **IEnumerable<T>** ile **ICollection<T>** ve **IList<T>** arasındaki hiyerarşi nedir? Her arayüzün sunduğu ek yetenekler nelerdir? Yalnızca okuma gereken bir senaryoda **IEnumerable<T>** döndürmenin neden daha iyi bir API tasarımı olduğunu açıklayınız.
- ◊ **Düşün:** Değiştirilemez koleksiyonlar (**ImmutableList<T>**, **ImmutableDictionary<K,V>**) ne zaman ve neden tercih edilir? Çok iş parçacıklı (**multi-threaded**) bir ortamda değiştirilemez koleksiyonların **lock** ihtiyacını nasıl ortadan kaldırdığını açıklayınız.
- ◊ **Çöz:** Jenerik bir **MinYığılı (MinHeap)** sınıfı tasarlayınız: ekleme, minimum değeri çekme (**extract-min**) ve minimum değeri gözetleme (**peek**) yöntemlerini uygulayın. **IComparable<T>** kısıtlamasını kullanın.
- ◊ **Çöz:** Bir metin belgesi içindeki sözcük frekanslarını tutan bir program yazınız (**Dictionary<string, int>**). En çok geçen 10 sözcüğü sıralayarak listeleyin. Büyük/küçük harf duyarsızlığını ekleyin.
- ◊ **Çöz:** Jenerik bir önbellek (**cache**) sınıfı tasarlayınız: maksimum eleman sayısı aşıldığında en eski eklenen eleman silinsin (FIFO). **Dictionary<K,V>** ve **Queue<K>** birlikte kullanınız.
- ◊ **Çöz:** İndisleyici kullanarak matris gibi çalışan iki boyutlu bir jenerik **Izgara<T>** sınıfı tasarlayınız: hem **[satur, sutun]** hem de **[indis]** erişimini desteklesin.
- ◊ **Çöz:** Thread-safe bir mesaj kuyruğu (**message queue**) sınıfı oluşturun: üretici (**producer**) iş parçacıkları mesaj eklesin, tüketici (**consumer**) iş parçacıkları mesaj çöksin. **ConcurrentQueue<T>** veya **BlockingCollection<T>** kullanın.